

The Cross-Domain Interoperability Framework (CDIF)

DDI-CDI Data Description Profile

This document is an extract from the full set of guidelines provided by CDIF. It describes the use of DDI-CDI expressed as JSON-LD which is recommended for use in cross-domain FAIR scenarios. Please note that SKOS is employed to describe enumerated value domains used to represent variables (as opposed to the native DDI-CDI classes). In CDIF this is covered by the “Controlled Vocabularies” profile. You will find this in the full CDIF document at: <https://doi.org/10.5281/zenodo.11236871> .

Alternately, the “CDIF Book” can be found at: <https://cross-domain-interoperability-framework.github.io/cdifbook/introduction.html> .

This document has three basic parts:

- Explanatory material (sections 6.6.1 to 6.6.4)
- Class diagrams and explanation (section 6.6.5)
- Properties and associations by class (Appendix 4)

6.6. Data integration

This section lays out the process of describing data to be disseminated in an ‘integration-ready’ form. The general intention is that domain standards containing much of the needed metadata can be mapped into the ‘integration-ready’ form for exchange across domain and infrastructure boundaries, and that this mapping can be done with an acceptable degree of effort, in some cases automated completely. Some examples of how these transformations can be achieved are provided. A further section explains how the production of integrated data sets can be described, although CDIF does not provide recommendations at this point for doing so.

This discussion is focused on data that can be represented in tabular formats with simple literal values. While this does not cover all the data we wish to describe, it represents a significant portion of it. Much of the data made available for reuse is expressed in CSV or similar, text-based formats, and these provide the initial focus for describing data. In future, this coverage will be expanded: this initial focus is intended as a useful starting point.

6.6.1. Introduction

Data integration is a task performed by researchers and others who wish to produce a unified data set which can be subject to analysis, or more easily used in analysis. Many of the decisions made during the integration of two unlike data sets depend on the analysis to be performed. For CDIF, the main goal is to

provide a description of data so that it is 'integration ready'. In a FAIR scenario, the user of the data is not a known quantity, and only they can make the decisions about how to integrate the data they are working with.

There is, however, a secondary scenario in which a data integration has been performed for presentation of harmonised data to a user. In this case, the fashion in which the data of interest has been integrated from various sources is a critical aspect and needs to be described, and the process steps recorded - this crosses into what we often think of as 'provenance' metadata, which covers a detailed description of the processing and transformations performed, along with information about the methods, etc. In this initial description, we do not attempt to provide this more-complete description.

Both of these scenarios require a wealth of metadata at a granular level if we are to maximise the automation of data integration functions. This is an important goal, as the cost of 'data wrangling' is high in terms of overall resources. While it is likely that data integration will always require some attention and input from the researcher, many of the necessary tasks are routine and can be automated if sufficient information is known about the data themselves. CDIF attempts to provide a sufficient level of metadata to support such automation.

Given this granular description of the data, it is possible to extend this information set to describe the mappings used to merge data sets. This information can be expressed at the level of the enumerations and types of data found in individual fields of the data sources, both in terms of their meaning (semantics) and in terms of their syntactic representation. In addition, other processes may be in play during a data integration. CDIF separates these different aspects, providing for a description of that individual data sets, in reference to the controlled vocabularies which provide the semantics and codes used in the data, and supplementing these descriptions with separate expressions of the mapping between the controlled vocabularies used, and with a more-complete description of the processes which were performed. Together, these aspects make up the provenance of the integrated data set.

The CDIF recommendations for data integration focus on two specific areas:

1. The description of data sufficient to make it 'integration ready', including references to the needed controlled vocabularies for describing field-level semantics and enumerated values;
2. The description of how two or more integration-ready data sets can be combined to describe an integration process, through the inclusion of mappings and process descriptions. (The recommendation for describing controlled vocabularies, mappings, and processes are treated separately: here we explore how these different metadata are used in combination to describe a data integration).

For the first scenario, we make specific recommendations about data description. The second area is one where more work is needed, but some progress has already been made. This description of data integration is a priority activity for future versions of the CDIF recommendations.

6.6.2. Problem statement

In a cross-domain scenario, it is necessary that a data resource, once located, can be loaded into the systems which will make it practically available for use. This is a metadata-intensive task, and one which has several dimensions. CDIF attempts to separate the metadata requirements in a fashion which allows them to be more easily satisfied.

The disseminator of data cannot know what structure will be needed by potential users, as their integration requirements and systems may be designed to work with data which is structured differently. The disseminator can, however, describe the data as they manage and present it, along with information about its logical contents. The user can then re-structure the data as needed for their own use, and do so programmatically. Sufficient metadata must be available to support this programmatic restructuring, without losing any of the information about the data - especially its links to semantic definitions.

The process for providing such detailed descriptions of the data can be broken down into a series of steps:

1. Identify the logical variables in the data, where each 'variable' measures a single characteristic of a single unit type, using a consistent set of values. The possible values must be enumerated or otherwise described in a detailed fashion. Representations must be able to identify domain-agnostic semantic descriptions for each possible value, and the variable definitions themselves must similarly be independent of any domain specificity.
2. Indicate how the logical variables fit into the structure of the file, by specifying also any 'presentational' variables (that is, variables which may be needed for structuring the file, rather than for describing the logical content of the data). Relationship between presentation and logical variables must be specified. The complete set of variables can then be described as a 'logical record'.
3. Describe which fields are used to identify the records (the 'primary key').
4. Describe the encoding of all variables physically present in the file, and how they are sequenced and stored for programmatic retrieval.

The description of variables includes an indication of where the semantics for both the field and the values come from. Because the semantic distinctions used in different domains can be very diverse, it is often not the case that these can be automatically subject to mappings or transformations. This step often requires the judgement of the user. Presenting the semantics used in the data — programmatically mapped/transformed to the extent possible — is often the closest we can come to a fully automated data integration.

Even so, being able to automate the structural transformation of data, accompanied by the needed semantics, with the roles they play vis-a-vis the data clearly defined, represents a major efficiency gain: manually performing these steps can be costly and time consuming.

6.6.3. Standards landscape

Identifying standards which will meet the requirements described above is a challenge. Many domain-specific standards have excellent descriptions of the data used within their domains, and some of these separate the semantic aspects of data description from the structural ones in the fashion described above. The number of candidates is small, however.

*CSV on the Web*¹ is one candidate, as is the W3C's *Model for Tabular Data and Metadata on the Web*². *Frictionless Data*³ provides a similar lightweight description of data. More metadata-rich models also exist, such as the SDMX⁴ Information Model and the RDF DataCube Vocabulary⁵ which is based on it, but these are largely limited to describing multi-dimensional data. Although the *DDI Codebook*⁶ and *DDI Lifecycle*⁷ specifications are very close to the CDIF requirements, they are also inherently bound to their intended use for describing social, behavioural, and economic (SBE) data. Only the related *DDI Cross-Domain Integration* (DDI-CDI) specification⁸ meets all of these requirements, largely because it was designed specifically to describe cross-domain data for the purposes of integration.

It is worth considering the strengths and weaknesses of each of these potential models, so that we can better understand the problem and the metadata needed to solve it.

We must have sufficient agreement between systems so that a requesting system can obtain and transform the data described, based on the disseminator's description of the data as understood within the distributing system, and to do so without benefit of human intermediation. Thus, we cannot safely rely on the conventionalized use of very flexible formats. On the other hand, we do not want to be overly restrictive in how data is described, as this will increase the barriers to use by disseminators.

This is a difficult challenge indeed, and it accounts for the fact that this type of ubiquitous, generic data description has been slow to emerge. To support cross-domain FAIR, as envisioned by CDIF, however, we must have a workable solution.

Consider the different specifications listed above. All of these provide a separation of concept definitions - semantics - from the structural description of data, a pre-condition for a useful cross-domain standard.

CSV on the Web is designed to add metadata to CSV tables, which is a use case not dissimilar to the one we address in CDIF. The problems arise when we consider the flexibility of the CSV format itself: there are no limitations in CSV regarding the logical organisation of the contents of the table: is each row a

¹ <https://csvw.org/>

² <https://www.w3.org/TR/tabular-data-model/>

³ <https://framework.frictionlessdata.io/docs/guides/describing-data.html>

⁴ <https://sdmx.org/>

⁵ <https://www.w3.org/TR/vocab-data-cube/>

⁶ <https://ddialliance.org/Specification/DDI-Codebook/2.5/>

⁷ <https://ddialliance.org/Specification/DDI-Lifecycle/3.3/>

⁸ <https://ddialliance.org/Specification/ddi-cdi>

unit being observed, and each column a characteristic of that unit? Or are the columns instead the units? This logical certainty is required. Although CSV on the Web allows us to add richness to the meanings of columns and rows of a table, it is in this sense too flexible to make a good foundation for the scenario addressed by CDIF. We can know what concepts are in play, and how rows and columns are organised, but we cannot understand how the presentation of the data — the table organisation — is *logically* structured. What roles do the concepts play? While this could perhaps be reverse-engineered, it is easier to explicitly state how the concepts logically relate, and how these in turn are presented in a tabular form. A similar critique can be applied to *Frictionless Data's* data description, and to the *Model for Tabular Data and Metadata on the Web*. These specifications are all designed to perform more limited tasks, and they do it well (they were all inputs in the design of the DDI-CDI model, in fact), but they fail to cover all the needed logical relationships, being restricted to tabular descriptions which combine logical and presentational aspects of the data.

SDMX and the *RDF DataCube Vocabulary* have a different issue. They have stronger formal models for how concepts intersect with data structure, and do not combine the presentational and logical aspects of data description in the same way, but they are limited because they insist on a multi-dimensional description of data sets. (It should be noted that while *SDMX* 3.0 has introduced a 'microdata' description feature, this is still new, and is not yet the version adopted by most implementations, nor is it the version on which the *RDF DataCube Vocabulary* is based.)

SDMX demands that the metadata description be provided using a model which may or may not be supported by disseminator's systems. Many data repositories do not manipulate data as multi-dimensional cubes, and lack the information needed to describe their data in this fashion. Further, *SDMX* uses a very disciplined cube definition: all data has regular dimensionality. In the real world, even many systems which are based on multi-dimensional models have irregular dimensionality and 'sub-cubes', which are not permitted in *SDMX*. Given that *SDMX* is an exchange model for official statistics, this disciplined approach is very reasonable, but is not appropriate for CDIF, where existing data stores must be described according to metadata which already exists.

Both *DDI Codebook* and *DDI Lifecycle* are excellent models in terms of being generic-but-concept-rich data descriptions, but they also lack the range of support needed for CDIF. In the SBE realm, data is overwhelmingly stored and processed as 'wide' data files: unit record data, where each record is a set of measurements or values about a single unit, one per row. (Admittedly, this is a simplification, as there can be hierarchical relationships between such records, but that is a side point for the purposes of this discussion.)

The *DDI Cross-Domain Integration* specification was developed exactly because it is increasingly common for other types of data to be combined in research projects with this traditional form of SBE data. Four different types are identified: wide, long, multi-dimensional, and key-value (the kind of data commonly found in 'big data' systems). Because they are described in a single model, intended to support transformations between these different structural types, the needed separation of logical and presentational aspects of the data was a necessity.

One might ask why a graph model is not used instead, as graphs do a complete separation of presentation and logical relations, and are absolutely flexible. (Indeed, the virtues of graphs are exactly why CDIF uses them for the purposes of structuring metadata.) It is not common practice to store or disseminate data in graph form today, for several reasons. Graphs are difficult to manage, especially when historically data has been managed in ‘data sets’ stored as files or tables in databases. While some domains are able to manage and disseminate data at a very granular level, this is certainly not true of all. Further, this type of system places demands on the scalability of systems, and the RDF solutions available today for disseminating graphs often fail when it comes to performance. For metadata, which tends to be smaller in size, this is less of an issue, But if a URI must be assigned to every single value in each organisation’s data repositories, the management burden quickly becomes a barrier. While such an approach may become feasible in future, it is not feasible across domains at the current time.

In DDI-CDI terms, graphs of data are essentially key-value systems: each data point has a unique identifier, allowing it to take its place within the graph. The DDI-CDI model accommodates this approach conceptually, as well as the way it has been implemented in big data stores, but this is not sufficiently common to be used as the basis for a more limited description of data for the purposes of FAIR data exchange and integration as intended here.

Ultimately, while DDI-CDI may seem to be a complex model, attempts to produce a ‘simpler’ model based on existing W3C specifications produced equal complexity, and were sufficiently extreme to no longer be extensions of another standard, but effectively new specifications in and of themselves. Using the DDI-CDI specification seemed to be the right approach for CDIF, which is intended to recommend other specifications, not to produce what are practically new ones, even if based on existing, simpler specifications designed for other purposes.

6.6.4. Implementation examples

CDIF is intended to leverage metadata which exists already with the systems and standards used within domains and communities of use, allowing it to be expressed as CDIF descriptions to better support reuse by others outside those communities or domains. Data description at a granular level sufficient to support interoperability and reuse is a metadata-intensive activity - one of the biggest challenges of FAIR implementation - so it is important to understand in practical terms how much effort will be required to support it. This section describes some of the early examples where the DDI-CDI profile used in CDIF has been implemented. (Some further thinking in this direction in the context of EOSC can be found in the paper ‘The Role of DDI-CDI in EOSC’.⁹)

Three examples are described here to illustrate the kinds of approaches which are anticipated. The first is a demonstrator tool which was designed for an exploration into how the SDG Indicators and their disaggregations could be easily described for integration. While the SDG Indicators are disseminated in SDMX, there was no similar publication mechanism for the disaggregations, although this topic is one

⁹ Gregory, A., Hodson, S., Wackerow, J. (2021). The Role of DDI-CDI in EOSC: Possible Uses and Applications. Zenodo. <https://doi.org/10.5281/zenodo.4707263>.

which receives a lot of attention.¹⁰ The tool was designed to take a simple CSV expression of the data and allow it to easily be marked up through addition of the needed additional metadata.

Our second example is one which was developed as part of the work in the WorldFAIR Social Surveys Case Study (WP06). In that domain, Stata and SPSS are very common software packages for data processing and analysis. The tools developed to mine the data and metadata out of these files and express them in DDI-CDI.

Our third example is a prototype developed to pull data and metadata from the SDMX Web services offered by ILOSTAT¹¹ and describe it in DDI-CDI format, with the idea that this could be used for further integration.

These examples show how existing resources can be used to generate those required to support the production of CDIF 'integration-ready' data with a reasonable amount of effort. Details on each example are provided below.

6.6.4.1. CSV-to-DDI-CDI Sample Generator example

This tool takes a CSV file (you can select one of the pre-loaded examples or upload your own) and generates the DDI-CDI metadata in the form of JSON-LD (as recommended by CDIF, although to an older version of the profile). Not all of the needed metadata is available in the CSV, and the tool provides an interface for a person to add the additional definitions and descriptions. It can be found at <https://ddialliance.github.io/ddi-cdi-sample-generator/>.

CSV is a format which is easily exported from many different systems, and is often used as an interchange format. In such cases however the structure and content is often required to be in some conventionalised form to support interoperability with other applications. This tool takes a very common style of CSV for aggregate data and allows it to be described with the needed additional metadata. The prototype tool runs as a stand-alone javascript application in a browser; there is no required server component. Thus, it is an extremely light-weight type of application.

The importance of this prototype is that it illustrates how an organisation which does not have sophisticated systems could manually produce CDIF-compliant data descriptions. While this would require some effort, it is not greater than what data archives and repositories do today when implementing standards such as DDI Codebook (the DDI Codebook expression of the metadata in XML is also produced by this tool, although not as useful for cross-domain FAIR purposes.) It does provide an easy way for developers of more sophisticated systems to generate sample code for testing and exploratory purposes. The tool here is not a production tool, but there is currently some discussion within the DDI Developer's Community of producing a production version.

¹⁰ See <https://unstats.un.org/sdgs/iaeg-sdgs/disaggregation/>

¹¹ <https://ilostat ilo.org/resources/sdmx-tools/>

6.6.4.2. SPSS/Stata-to-DDI-CDI Converter example

This is another exploratory prototype, developed by staff at Sikt, who are active both within WorldFAIR and in the DDI-CDI Working Group. This prototype will be a deliverable from the Social Sciences Case Study of WorldFAIR. It produces the XML expression of DDI-CDI, but this could as easily be rendered in JSON-LD, as per the CDIF recommendations (the two syntax representations are based on the same model, being generated programmatically from it.) It can be found at <https://ddi-cdi-converter-app.azurewebsites.net/>.

This tool also provides a way for a person to manually add metadata which is missing from the SPSS or Stata files. Unlike the CSV converter, this tool is server-based, and supports the description of Wide Data Sets, rather than Dimensional Data Sets. (The two are quite similar, however).

This is also a prototype, but it illustrates the way in which data archives in such domains as Social Science and Public Health — which often have data in SPSS and Stata formats — could produce ‘integration-ready’ descriptions of their data holdings to support cross-domain FAIR use, in line with the CDIF recommendations.

6.6.4.3. ILOSTAT SDMX-to-DDI-CDI example

The Statistical Data and Metadata Exchange (SDMX) standard is used widely within the official statistics community for the publication of aggregate dimensional data (and related metadata) on a wide range of topics. It is concept-rich, and is accompanied by a body of harmonised concepts which support data exchange. Moreover, SDMX provides standard application programming interfaces (APIs) as RESTful services.

Within the international statistical system there are a very large number of important aggregate dimensional data sets that are readily available in SDMX. The SDG Indicators are a primary example. Similarly, the International Labour Organization (ILO) publishes statistics on labour using SDMX, covering a wide range of data used for policy at the national and supra-national levels, as well as forming a large body of important reference data for scientific research.

While SDMX is excellent for structured data exchange, and includes metadata standards that are robust within the context of aggregate statistical data, it may not fully address the needs for integrating data across different domains and formats, and its metadata scope may be limited when dealing with more detailed or domain-specific information. Moreover, SDMX's standardised structure, while beneficial for consistency, can sometimes be rigid when attempting to extend beyond its predefined concepts and dimensions.

On the other hand, DDI-CDI is designed for cross-domain data integration, facilitating the combination of data from various sources and formats. It supports detailed data documentation and the use of standardised metadata, which enhances data interoperability and usability in broader contexts, including scientific research. In addition, DDI-CDI can describe data at a more granular level, including microdata, longitudinal studies, and complex datasets. It is designed to be more flexible and extensible,

allowing for the inclusion of additional metadata elements and the adaptation to various data types and structures, making it suitable for integrating statistical with non-statistical data.

This section presents as an example a prototype developed at the United Nations Statistics Division of the UN Department of Economic and Social Affairs, exemplifying how the standard SDMX APIs could be mined to describe dimensional statistical datasets using for statistical data and their related metadata, which could then be transformed them into a minimum DDI-CDI profile, with the objective of facilitating their JSON-LD for further use in further data integrations.

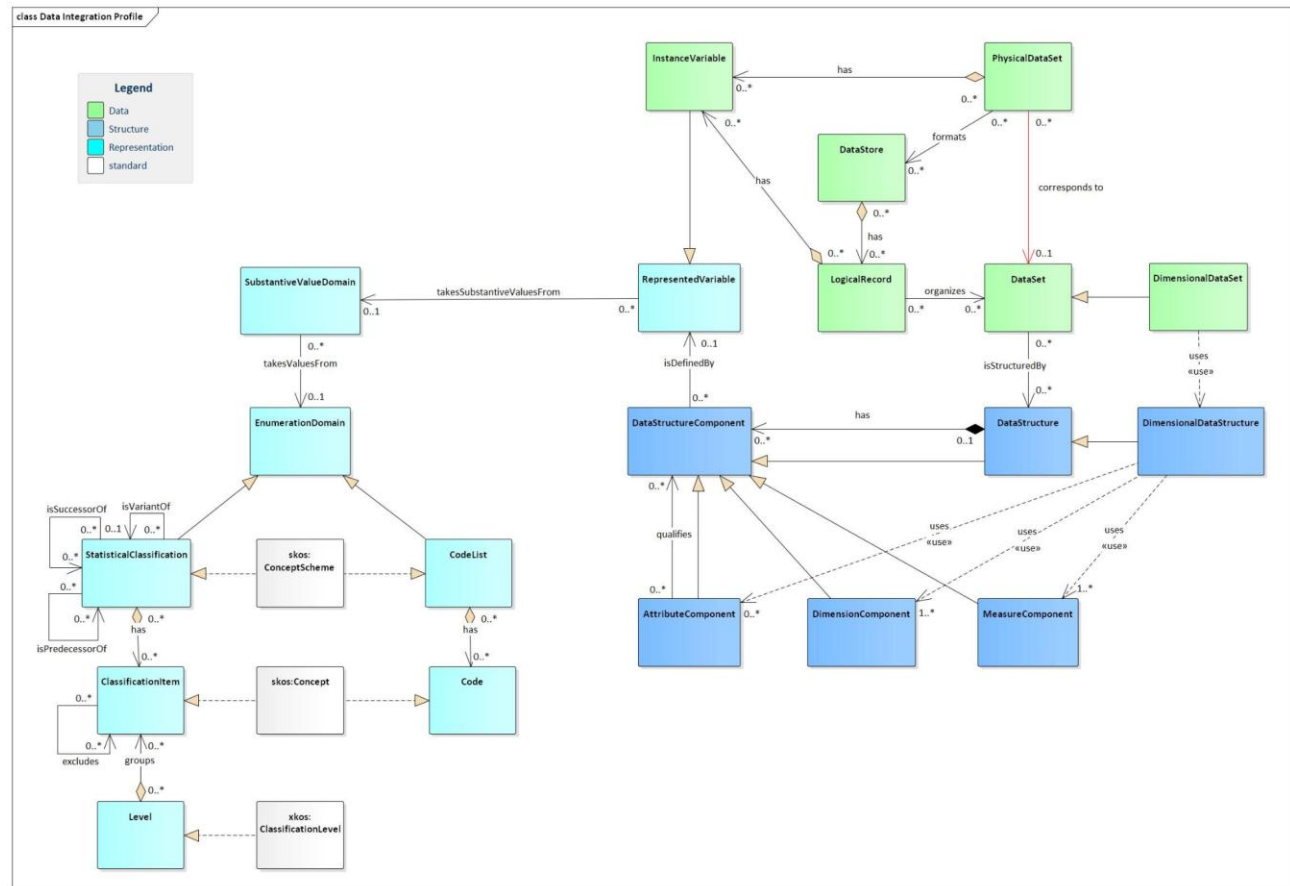


Figure 12. Prototype of a minimum DDI-CDI profile for data integration purposes.

This example is more fully documented than the other examples, with a tutorial available alongside the code in a Jupyter Notebook:

- Github link: https://github.com/Cross-Domain-Interoperability-Framework/undata_example_cdif
- ILO Jupyter Notebook link: https://github.com/Cross-Domain-Interoperability-Framework/undata_example_cdif/blob/main/src/cdip1_ILO_example-v2.ipynb

SDMX is used widely within the official statistics community for the publication of aggregate data on a wide range of topics. For this data, SDMX is the standard used both for data and metadata. Primarily

designed to describe aggregate statistics, SDMX is concept-rich, and is accompanied by a body of harmonised concepts which support data exchange. SDMX provides standard application programming interfaces (APIs) as RESTful services.

The SDG Indicators are a primary example of this type of data, but within the international statistical system there are a very large number of important data sets which are of a similar type. The International Labor Organization (ILO) is responsible for the publication of statistics on labour within a system that covers a wide range of data used for policy at the national and supra-national levels, as well as forming a large body of important reference data for scientific research.

This prototype takes data from the ILO implementation of SDMX Web services as an example, but this approach could be easily applied to a large number of other SDMX-using organisations within the international statistical community (Eurostat and the European Central Bank are also major users of SDMX, for example.)

The implications for a cross-domain FAIR scenario here are large: the body of official data, used as a reference point in many scientific domains, can be easily made available for integration with other research data. The SDG Indicators - one of the many data sets available through SDMX APIs - would become available for use not only on their own, but in combination with any other data as the basis of integrated data sets for analysis.

6.6.5. Describing data to make it 'integration-ready'

6.6.5.1. CDIF data description steps

This section explains how to describe a data set or a service providing data so that it can be used for integration and similar purposes. CDIF uses a subset of the classes in the DDI-CDI specification to do this. We will introduce the classes needed for each step, with more detail provided on the full set of needed properties in Appendix 4.

For a data set — that is, a static set of data — there are four steps. For a service, where the structure and physical format of the data will depend on the service, the last two steps are not required.

The steps are:

1. Describe the Data Set or Service;
2. Describe the Variables;
3. Describe the Data Structure;
4. Describe the Physical Format of the Data.

6.6.5.1.1. Describe the data set or service

The class used here will depend on what is being described. For Data Sets, the `cdi:DataSet` class is used. For services — that is, queryable sources of data — the `cdi:DataStore` class is used. The difference between them is that a Data Set object in DDI-CDI is static, with a fixed set of records (similar to what is seen in DCAT for a Distribution), a Data Store is more flexible: while it always contains the same logical

records, their expression (and the set of units measured) may change. These can be seen in the diagram below:

DESCRIBE THE DATA SET OR SERVICE

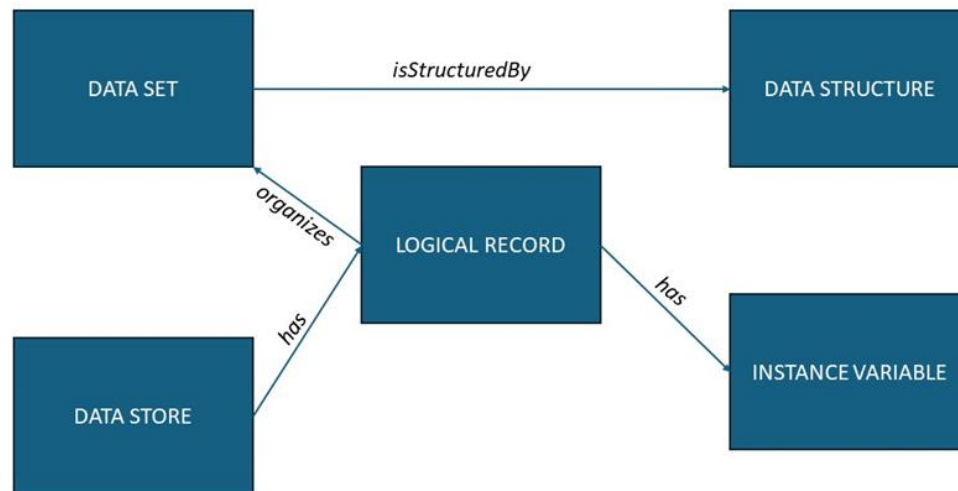


Figure 13. Describe data set or service.

A Data Set and a Data Store both have Logical Records: arrays of variables which make up the records in the data. The Logical Record has a set of Instance Variables, described more completely in the next step. These are the different fields which are used to hold the data (like the columns in a SQL table). A Logical Record is not a physical arrangement of the data – it is simply a set of related fields, all of which will potentially exist in a single record.

An additional class may be associated with the Logical Record if desired: a Population, Universe, or Unit Type may have an *isDefinedBy* association with it. These classes are concepts which can be used to associate the Logical Record with the thing it measures. The use of this class is always optional: we will describe these when we talk about variables, below.

A Data Set has a Structure, which can be described — a Data Store (a 'service') does not have such a fixed structure, as this will generally be dependent on how it is queried. The Structure description varies according to what type of data set is being described — whether it is organised as a 'wide' table, as a 'long' stream of data, or as a cube. Each of these types of data structure will be described separately.

While not shown in this diagram, the variables will be used as 'components' in the description of the data structure. A component is the role played by a variable in the context of a specific data structure: it is an identifier? A measure? A descriptor? The use of components for different kinds of data structures is detailed for each type of data structure in turn, as these are discussed below.

6.6.5.1.2. Describe the variables

In order for data to be easily reused, it is important for both users and machines to have details about the variables themselves. In DDI-CDI, there are several types of variables, each performing a different function. (This is termed the ‘variable cascade’, and may be of interest, but is out of scope for our immediate purposes.) Only two of the DDI-CDI variables are used here: the `cdi:InstanceVariable` class and the `cdi:RepresentedVariable` class.

The Instance Variable is the variable as it is used in the context of a Data Set. The Represented Variable is a variable used as a template across multiple Data Sets or Data Stores. It is worth considering the value of providing this additional metadata around Represented Variables.

To describe a single data set, we do not need to provide information on Represented Variables - the fact that a variable is reused elsewhere is not necessary for someone to process the single data set. However, in cases where a data set re-uses a variable to promote comparison between data sets, this becomes very important. In many scenarios, the same variable is used in many different data sets to provide a point of comparison or integration. A typical case is that of repeat data collection across time, where we want to take a series of measurements to produce a time series.

Often, although the measurements are identical, they are given different names for the purposes of data management, typically by appending the time of data collection (such as a year) or a number indicating the wave (e.g., a repeated measurement of education level might have variables ‘educ_level_95’, ‘educ_level_96’, ‘educ_level_97’, and so on). In such cases, a Represented Variable can be extremely useful because it indicates without any ambiguity that two variables are in fact the same. This information can be very valuable to users performing integration or harmonisation of the data, or producing their own times series from it.

The DDI-CDI model uses Represented Variables when describing structures, and for many other purposes, but uses Instance Variables when describing physical layout of the data records. This can seem confusing. In the DDI-CDI model, we can avoid the complexity of this arrangement, however, because according to the DDI-CDI model the Instance Variable is a sub-class of Represented Variable, meaning that any given Instance Variable can also be treated as a Represented Variable, and inherits all its associations and properties.

For our profile, Represented Variables are always optional. When present, they should be declared *in addition* to Instance Variables, and the Instance variables which are based on them should have a *uses* association declared with the corresponding Represented Variable. Their representations will always be the same. In the diagrams, Represented Variable is presented in grey, to indicate this special, optional status.

In the diagrams below, we always show the Instance Variable being used, as that is required. Note that if Represented Variables are present, these may be the ones which appear in the full DDI-CDI model, as for describing the representations of variables and their use as components in structures.

The diagram below shows the basic class structure for variables:

DESCRIBE THE VARIABLES

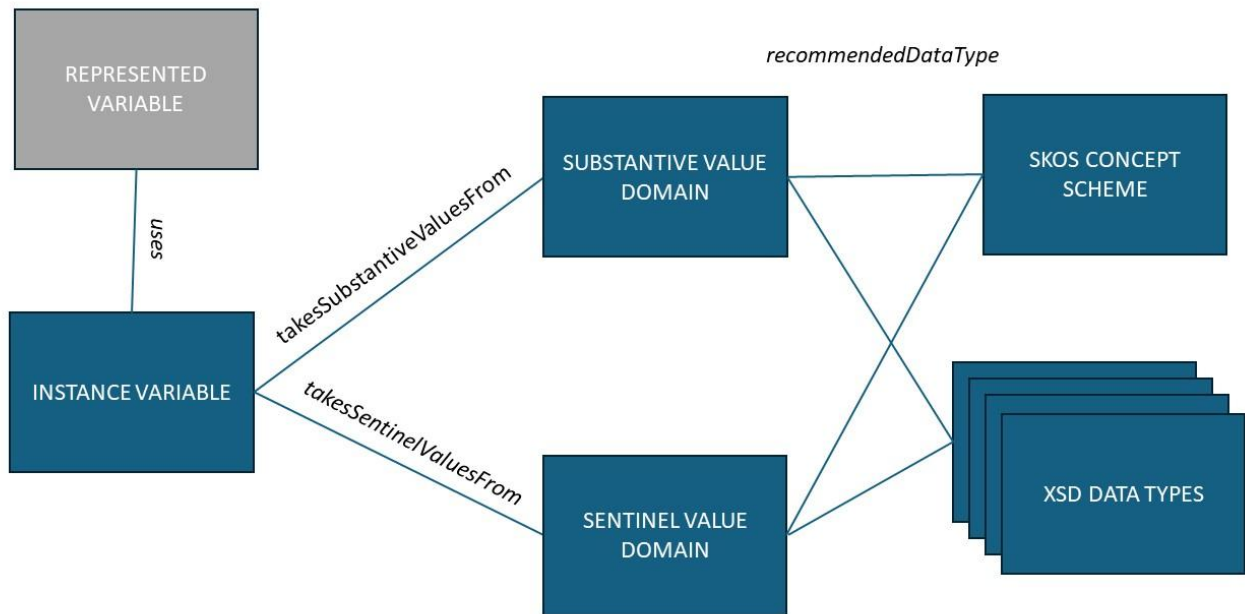


Figure 14. Basic class structure for variables.

Any given variable has a set of possible values, each of which comes from either a Substantive Value Domain (the meaningful values) or a Sentinel Value Domain (the flags for things like “missing” values). It is required to have an instance of the `cdi:SubstantiveValueDomain` class when describing a variable. It is not required to have an instance of the `cdi:SentinelValueDomain` class, although it is recommended if these are present in the data, as this enhances the practical reusability of the data. It is always the case that all possible values for the variable be described in one or the other of the value domains, however.

Both the Substantive Value Domain and the Sentinel Value Domain take their data types from either a reference to a `skos:ConceptScheme` (for enumerated values) or an XSD data type. The list below shows the valid XSD data types (XSD types available in RDF – see https://www.w3.org/2011/rdf-wg/wiki/XSD_Datatypes):

`xsd:anyURI`

`xsd:base64Binary`

`xsd:boolean`

`xsd:byte`

`xsd:date`

xsd:dateTime

xsd:decimal

xsd:double

xsd:float

xsd:gDay

xsd:gMonth

xsd:gMonthDay

xsd:gYear

xsd:gYearMonth

xsd:hexBinary

xsd:int

xsd:integer

xsd:language

xsd:long

xsd:Name

xsd:NCName

xsd:NMTOKEN

xsd:negativeInteger

xsd:nonNegativeInteger

xsd:nonPositiveInteger

xsd:normalizedString

xsd:positiveInteger

xsd:short

xsd:string

xsd:time

xsd:token

xsd:unsignedByte

xsd:unsignedInt

xsd:unsignedLong

xsd:unsignedShort

The definitions of variables are described in Appendix 4, below — these are provided as literal definitions but can also make reference to concepts defined elsewhere. Note that some variables may hold more complex values such as arrays. We do not cover this case in the current profile, but this is a subject which will be addressed in future.

It is also possible to instantiate classes which represent the subject of the data. This is possible at both the variable level and the Logical Record level. These classes include cdi:UnitType, cdi:Universe, and cdi:Population. A Unit Type is the most general class: it differentiates the top-level category of entity being observed, and is often implicit in the more-detailed classes (Universe and Population). Typical Unit Types are ‘person’, ‘organism’, ‘chemical compound’, ‘water’, and similar very broad categories. While conceptually always present, it is not always useful to describe these, as they are often implicit in the more-detailed (and more useful) Universe and Population descriptions. A Universe is a qualified Unit Type, but without specifics about time and place. For a Unit Type of ‘person’, I might have ‘student’ as a Universe. For a Unit Type of ‘organism’ I might have ‘dipterids’ as a Universe. A Population further qualifies the Universe by specifying time and place. These classes are instantiated with formal, textual definitions, which may be included as literal or by reference to an external source.

The diagram below shows how the variables and Logical Records can be associated with Unit Types, Universes, and Populations. The use of Unit Type, Universe, and Population is always optional, but may be useful, especially when variable-level searches are being made across broad sets of data for comparable measures on comparable populations.

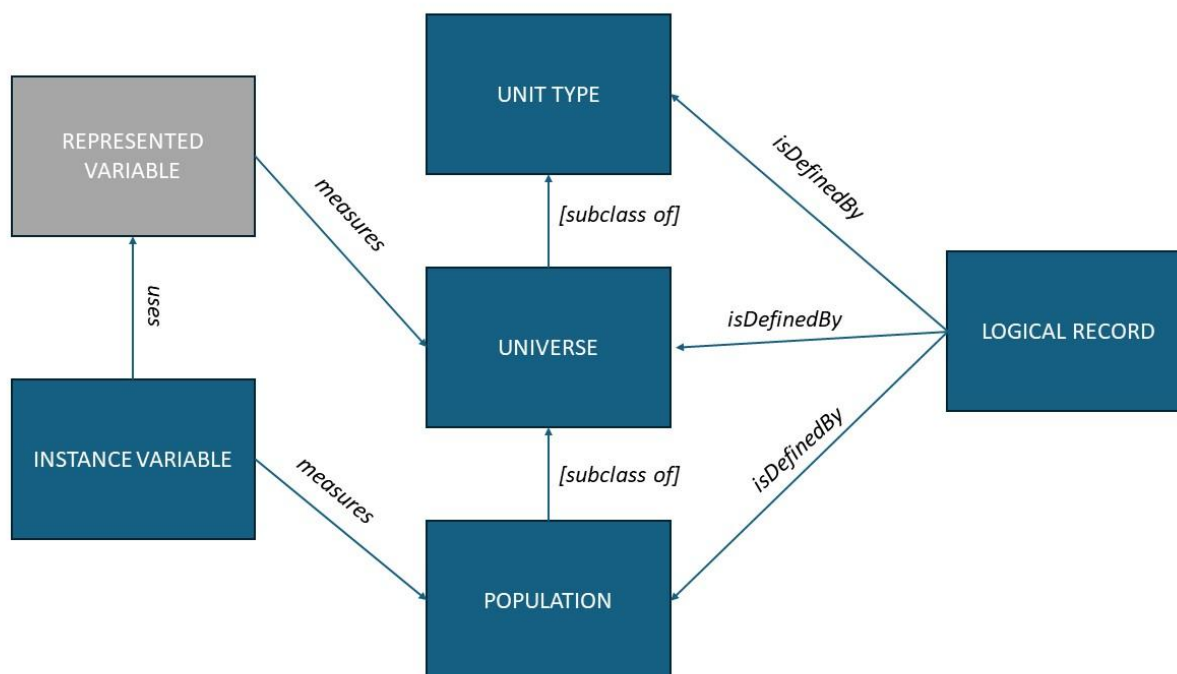


Figure 15. Unit, universe, and population.

6.6.5.1.3. Describe the data structure

For Data Sets (which always have a fixed structure – and these may include the results of querying services, with the Data Structure generated when the query is executed along with the resulting data), DDI-CDI gives us a model for describing the details. There are three types of Data Set: Wide, Long, and Dimensional. The descriptions of these are very similar, differing mainly in the way they identify specific data cells.

Variables are described separately from the Data Structure, because variables can be re-used across many different structures. I could, for example, have a ‘Nation’ variable which is always represented with a two-character ISO Country Code. This variable might be reused across a wide body of different data sets (think of data in the SDG Indicators and the Minimum Set of Gender Indicators - both have such a Country variable, and share some others, but are mostly different structures). The role of a variable within a specific Data Structure is represented by the `cdi:Component` class. There are several specific types of Components: identifiers, measures, attributes, and so on. These are described as appropriate for each of the three types of data structure.

Most data sets can be described as Wide Data Sets in DDI-CDI – these are those cases where each row in a tabular view of the data is a record, and each column is a single variable. (When in doubt, this is the place to start in most cases.) Long Data Sets are often seen in cases where there is a stream of data, and the measured values are accompanied by another field which tells you which type of measurement they are. This case is important, although probably less common. Dimensional Data Sets are ‘cubes’ where

the data points are addressed with the coordinates of a matrix, along different dimensions. This is a very common case for aggregated data.

6.6.5.1.3.1. Wide Data Structures

To describe a Wide Data Structure, each variable (here we use Instance Variables, acting in their capacity as Represented Variables) is assigned as defining a specific type of Component. In Wide Data Structures, the only Component classes we need are Identifier Components (cdi:IdentifierComponent), Measure Components (cdi:MeasureComponent), and Attribute Components (cdi:AttributeComponent). The diagram below shows how these and other classes are related.

Taxpayer ID	Year	Taxes Paid
003-49-6574	2001	2400.00
003-50-1234	2011	3600.00
009-42-6324	2021	2100.00

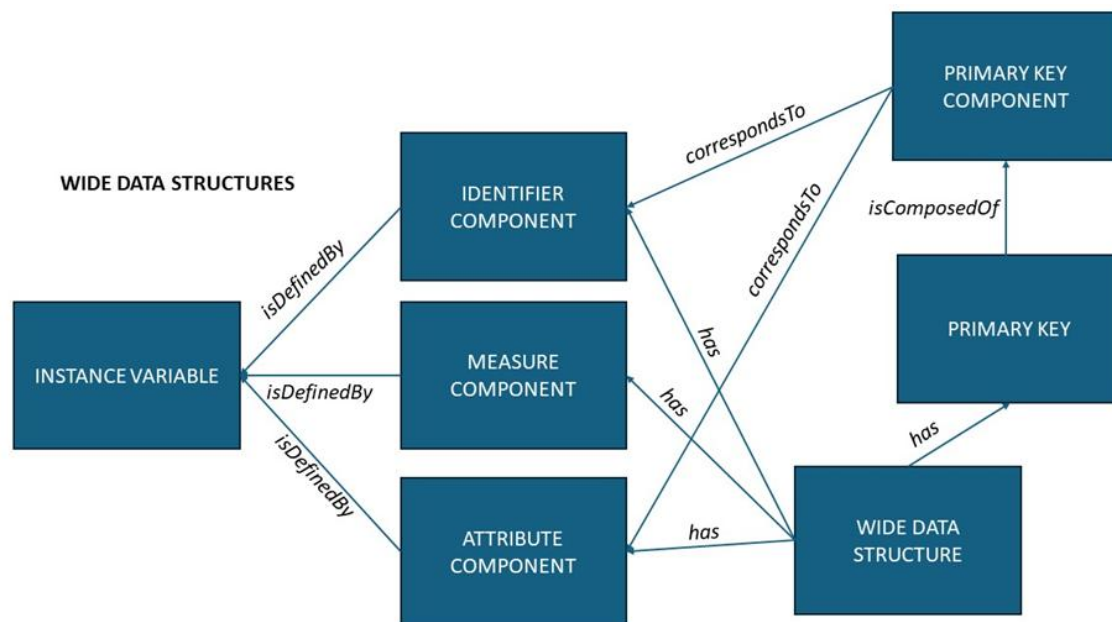


Figure 16. Wide data structures.

Measure Components hold the observed or measured values in our data. Identifier Components hold specific identifiers assigned to the cases in our data: each record is about a single 'case', and these are sometimes assigned identifiers to distinguish them.

As in **the example provided to the right**, this could be a tax-payer ID for a person, for example, or might be a randomly assigned number in the case where the cases have been anonymised to protect their identities.

It is sometimes the case that an identifier Component is sufficient to disambiguate all the records in the data set: Identifier indicating the row/case, and the variable telling you which value in that record. Often, however, there will be additional fields which are needed (like a timestamp, in the case where the same case was measured/observed on more than one occasion). Attribute Components contain additional information about the measurements/observations which are applicable to each case. Typical Attribute Components are time stamps, geographical coordinates, statuses, or other data related to the context of the measurement or observation.

The Primary Key is the set of Components needed to uniquely identify each record in the data set. It is not typically the case that the Measure Components form part of the Primary Key.

Long vs. Wide Data Sets: If a Measure Component in a data set cannot be clearly associated with a single variable, but relies on the value of another variable to indicate which variable it measures, then you have a Long Data Set, and not a Wide one.

To illustrate the difference, some examples may be helpful. The table below is a simple example of a typical Wide Data Set:

Here, the Taxpayer ID is an Identifier Component (it gives the unique ID of each case), the Taxes Paid column is a Measure Component, and the Year is an attribute. If the cases are restricted to one entry per taxpayer, then the Primary Key will need only the Taxpayer ID field, but if (as is likely, given the need to pay taxes annually) there was more than one entry per taxpayer, then the Primary Key would also need to contain the Attribute Component Year. Compare this to the example in the *Long Data Structures* section below.

6.6.5.1.3.2. Long Data Structures

Long Data Sets are typically coming from sensors, administrative registers, or systems which contain 'spell data'. These forms of data are structured differently than what we see in most SQL-based systems, and they have a different relationship to the logical contents of data.

A second example can help to illustrate the difference between Wide and Long Data Sets. Long Data Sets use additional Components, as they have a more complex relationship to logical variables.

The table to the right shows a simple Long Data Set. Here, we have an Identifier Component (Patient ID), a column giving us the test performed (Test), a column with a measurement (Reading), and a column with a unit of measure for the reading (UoM). The UoM column is an Attribute Component: each entry provides a value that represents a Unit of Measure, and we can easily describe a 'Unit of Measure'

variable (even though that can be inherent in the definition of a variable as is often seen in data sets today.)

The Reading column is more problematic: each cell contains a measurement, but they are different types of measures. In order to understand the measurement, you have to consult the Test column, which tells you whether it is a pulse, a temperature, or a weight. This dependency is the characteristic feature of Long Data Sets.

Patient ID	Test	Reading	UoM
GX6574	Pulse	80	BPM
GX6574	Temp	37.00	Celsius
GX6574	Weight	75.00	KG

When we describe a Long Data Structure, it is important to distinguish between the *logical* content and the presentation of it. Consider this re-structuring of the information shown in ***the example to the right***.

The values in the Test column have been broken out into separate columns which cleanly align with the measurements being taken, containing the values formerly held in the Reading column. The UoM column is now part of the definition of each of these variables, and so does not need to be included here. Logically, however, this is the same data as that shown above. (This re-structuring is illustrative – we are not recommending that Long Data Sets be re-structured as Wide Data Sets for publication.)

Patient ID	Pulse	Weight	Temp
GX6574	80	75.00	37.00

In order to describe my Long Data Structure, I need to define the *logical* variables, as seen in this re-working of the table, so that each variable can have a single, ‘clean’ definition, containing a single type of measurement. Each becomes an Instance Variable in my metadata. I would then go back and identify the additional variables needed to support the original presentation of the data: the Test column, the Reading column, and the UoM column.

UoM would be described as an Instance Variable, but Test becomes a special type: a Descriptor Variable (cdi:DescriptorVariable), and Reading becomes a Reference Variable (cdi:ReferenceVariable), which is the name for a variable which combines measurement values from several logical Instance Variables. These new types of presentational variables have different component types. The UoM column is still an Attribute Component, but the other columns introduce new ones: a Variable Descriptor Component (cdi:VariableDescriptorComponent), which is defined by a Descriptor Variable (Test), and a Variable Value Component (cdi:VariableValueComponent), which is defined by a Reference Variable (Reading).

Thus, my resulting metadata will have descriptions of both my logical (Patient ID, Pulse, Weight, Temperature) and presentational (Test, Reading, UoM) variables. The Components in my Long Data Structure would reference the presented variables: Patient ID (an Identifier Component), Test (a VariableDescriptorComponent), and Reading (a VariableValueComponent) but the logical variables in turn would all be referenced from the VariableDescriptorComponent and associated classes, as described below.

VariableDescriptorComponents have a *refersTo* association with a VariableValueComponent (in this case, Test will associate itself with Reading in this way).

Descriptor Variables – the only variables which act as Variable Descriptor Components – do not have the same type of representations as other variables. Instead, they will always take their values from a Descriptor Value Domain (cdi:DescriptorValueDomain). This is a class which associates with a set of Descriptors (cdi:Descriptor), which are codes that refer to logical Instance Variables using an *identifies* relationship. The code in the Descriptor Component indicates which Instance Variable the value provided in the corresponding Reference Variable should be associated with. Thus, in our example above, when the :Test” column has a value of ‘Temp’, we know that the value in the “Reading” field is a measurement of temperature, associated with the temperature *logical* variable.

Reference Variables always have a representation which is the super-set of the values of the variables for which they can hold measures. They are thus often declared as generic types such as XSD string. The specific types of any value can be determined from the description of the associated Instance Variable (the logical variable) and do not rely on the Reference Variable. Figure 17 shows how these new presentational classes are connected.

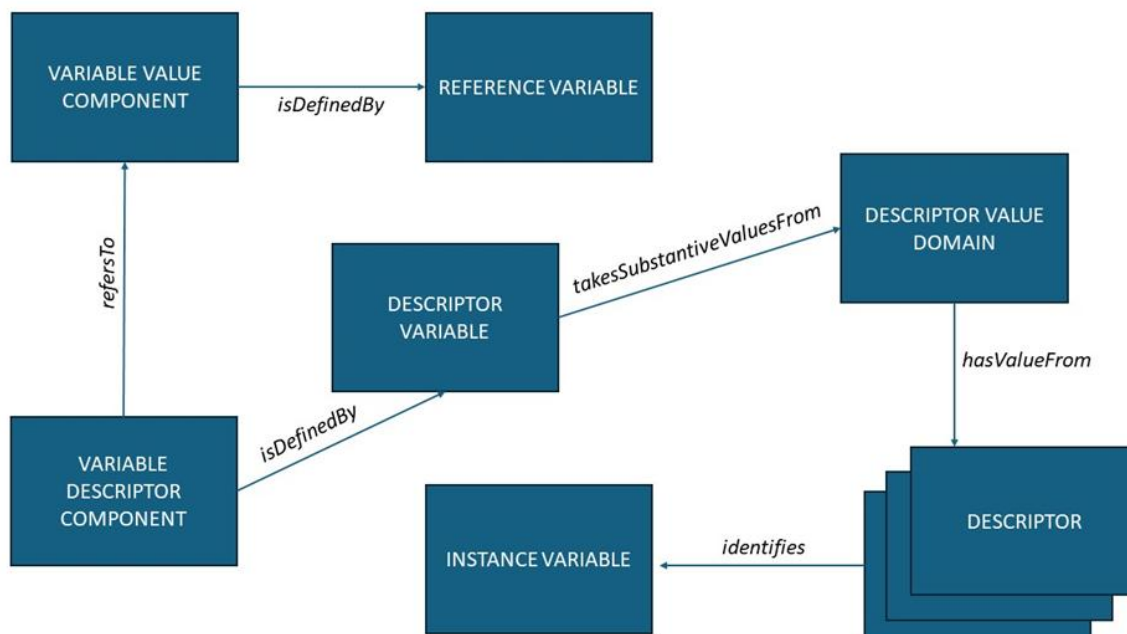


Figure 17. Long data structures - presentational elements.

The Primary key will be assembled from the *presentational* variables, however: in this case, a combination of the Patient ID and the Test.

The point of having this additional structural metadata for a Long Data Structure is that by providing logical Instance Variables, we are able to reassemble the values in the data set according to other structural arrangements (typically Wide or Dimensional), and to do so programmatically. Although we

could have described the example table as a Wide Data Set, the actual variables in the data could not then be re-arranged or re-used: they would be specific to the structure of the data set they appeared in.

6.6.5.1.3.3. Dimensional Data Structures

Dimensional Data Sets have a Dimensional Data Structure which is similar to the Wide Data Structure, but has a different kind of Primary Key. The diagram below shows the set of classes which are needed.

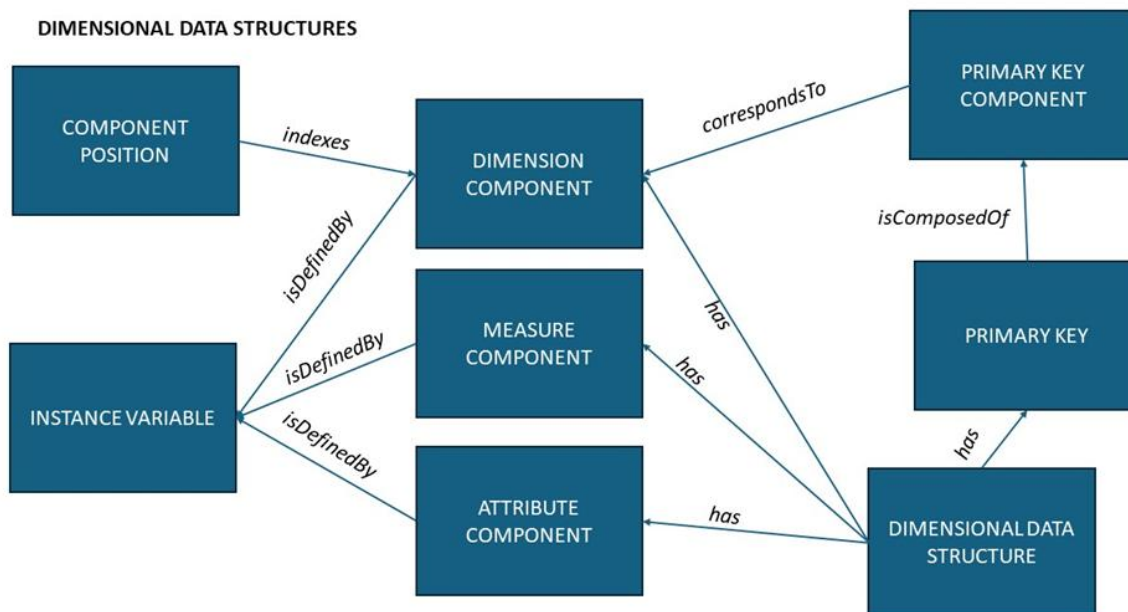


Figure 18. Multidimensional data structures.

In describing a Dimensional Data Structure, our Primary Key is only made up of Dimension Components. Taken together, these address not an entire record, but an individual cell with a cube – a multi-dimensional matrix. That cell holds the value of the Measure Component. Attribute Components are associated with the Measures at the cell level, not at the level of an entire record.

The exception to this is if a ‘cell’ has more than one Measure Component, in which case the set of Measure Components requires further disambiguation. This can be specified by qualifying the Primary Key values with the Measure Component/Variable. (It is recommended that only a single Measure Component be used if practical — cells should, if possible, hold simple values such as a string/code, a date-time, or a number.)

In some cases — notably in the DataCube Vocabulary from W3C and the SDMX specifications on which it is based — the dimensions in a key are ordered. In DDI-CDI, specifying an order is done using the Component Position (`cdi:ComponentPosition`) class. (In fact, you are always allowed to use

cdi:ComponentPosition for any type of Data Structure, but in this profile we only recommend it for Dimensional Data.)

The table, ‘Education Level of Belgian Residents’, provides a simple example of a Dimensional Data Set, and how its structure can be described.

In this example, the first four columns act as dimensions for addressing the value given in the last column. There would be five Instance Variables described: Year, Degree, Province, Age, and Percent. The representations of these variables would be enumerated using SKOS Concept Schemes (see above) except for Year, which would be an XSD gYear, and the Percent variable, which would have a numeric type.

Education Level of Belgian Residents

Year	Degree	Province	Age	Percent
1986	BA	Antwerp	21-30	.25
1986	PhD	Brabant	31-40	.10
1987	BA	Limburg	21-30	.36

Year, Degree, Province, and Age would all be used to define Dimension Components, which could be indexed using instances of the Component Position class, assigning them a ranking from 1 to 4 (ordered low-to-high). Taken together, these four Dimension Components would form the Primary Key. Percent would be a Measure Component.

6.6.5.1.3.4. Notes on Data Structures and reuse

It is evident that many different types of Components are shared between different Data Structures. It is important to understand that although this is true – and that any given set of data may be organised according to more than one type of Data Structure, the instances of Components themselves are *not reusable* by definition. A Component is the specific role played by a variable in the context of a data structure. Each Component appearing in a Data Structure is specific to that Data Structure, and should have its own identity. The variables are the reusable form of the information in describing the same thing used across different structures.

Remember, too, that while the DDI-CDI model uses Represented Variables – the reusable part of a variable description – in modelling Data Structures, for the purposes of CDIF we rely on the fact that Instance Variables are a subclass of Represented Variables, and can thus be used *as* Represented Variables. (This is expressed with the `rdf:instanceOf` relationship on the instances of `cdi:InstanceVariable` – see Appendix 4, below.)

6.6.5.1.3.5. Describe the physical format of the data

DDI-CDI separates the logical and physical organisation of data. The way that data is stored in a system or recorded in a file has no necessary relationship to the way in which the data is logically structured.

In this initial version of CDIF, we are only describing data that is expressed in a textual format such as a CSV file, fixed-width tabular files, or the query result from a relational system. (This is a limitation which

will be addressed moving forward.) Further, the assumption is made that any Data Set uses only one structure for all of its records, and that these are organised in a uniform way.

These recommendations pertain to data that can be represented in a tabular data structure, packaged in datasets that consist of a set of records that all have the same set of fields. Figure 19 shows the classes which are needed for describing the physical encoding of the data.

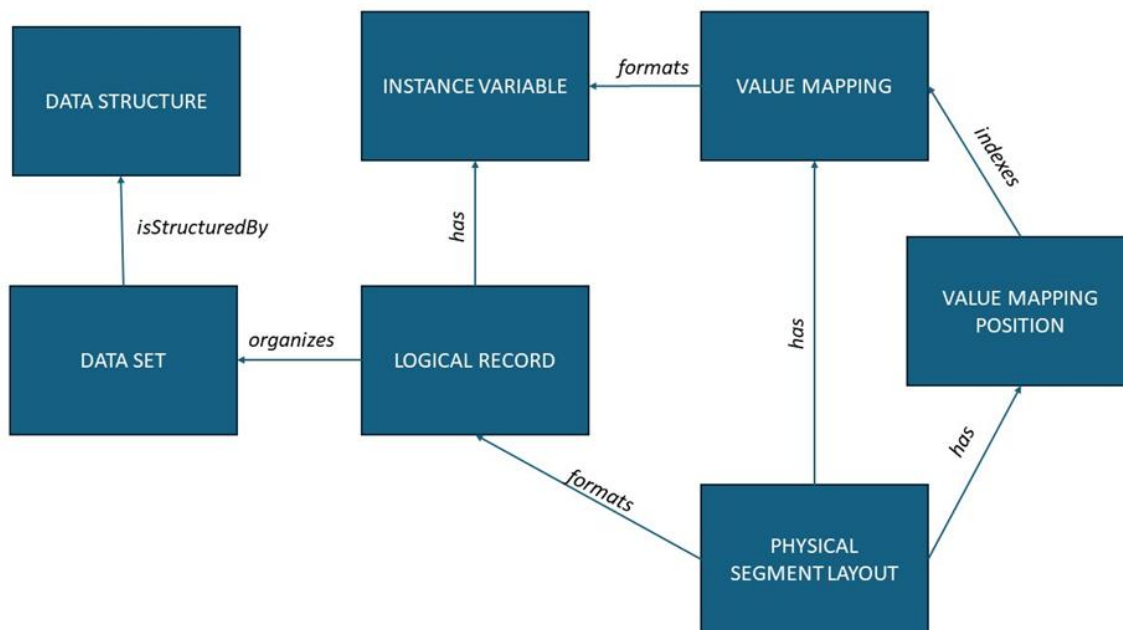


Figure 19. Describing physical layouts.

The Logical Record (`cdi:LogicalRecord`) references all of the Instance variables used within the Data Set. A Physical Segment Layout (`cdi:PhysicalSegmentLayout`) describes the way in which that Logical Record is expressed in the physical file. It has a set of Value Mappings (`cdi:valueMapping`) instances and corresponding Value Mapping Position (`cdi:ValueMappingPosition`) instances, which provide the links between the physical layout and the values of the Instance Variables. Physical Segment Layout instances contain much of the information needed by machines to read the data (i.e., character encodings, delimiters, line-end characters, etc.).

(Note that the *formats* relationship between Value Mapping and Instance Variable has been collapsed from what is presented in the DDI-CDI model by omitting the intervening Data Point, as Data Points are not instantiated in this profile.)

6.6.6. Describing the integration of data sets

This version of the CDIF recommendations does not contain a full profile for the description of data integration, but this topic has received a lot of attention from the working group. An exploration into different data integration scenarios has been conducted, notably an effort to integrate data from ILO,

the World Health Organization (WHO), and the SDG Indicators, with a goal of publishing this data into Google Data Commons and the knowledge graph model used there. This work is on-going, but it has demonstrated the set of metadata needed to fully describe a data integration.

This set of metadata requirements can be summarised as follows, based on the assumption that there is access to the data being integrated:

1. Detailed data description (variable-level)
2. Structural metadata of the data sets
3. Enumerated values (codelists and classifications)
4. Mappings between sets of enumerated values
5. Processing description, indicating how mappings were implemented in transformations, and what other operations were performed for data integration

While this seems like a daunting set of information, the exploratory work has shown that if the metadata are available in a sufficiently detailed form, then the actual integration itself is straightforward.

Appendix 4. Metadata for making data ‘integration-ready’: DDI-CDI classes and properties

This appendix lists the classes discussed in the text above, and indicates which properties must or may be used when these classes are instantiated.

It should be noted that while the Data Set and Data Structure classes appear in the DDI-CDI model, and are used in the diagrams above, these are never directly instantiated. Instead, their sub-classes will be used: Wide Data Set/Wide Data Structure, Long Data Set/Long Data Structure, and Dimensional Data Set/Dimensional Data Structure. For this reason, Data Set and Data Structure are not listed here.

The identifiers of each metadata declaration (or instance) are assumed to be the URI used in the JSON-LD. If other identifiers are to be associated with the instances, all of the classes listed can also use an identifier property as per the DDI-CDI specification. This is not required, however, as it is assumed that consumers of CDIF metadata will typically not be interested in non-public identifiers of this type.

Properties and associations are listed as “Required” or “Optional”, followed by “Multiple” if more than one is allowed. Language-specific strings are allowed to be multiple, specific to language-locale pairs, using language maps as described in Example 71 of the JSON-LD specification¹².

All metadata instances will have @type referencing the DDI-CDI class, given in the “Type” field. Some instances will declare multiple types where appropriate. The classes are grouped by type, largely in the order they are mentioned in the steps above.

Note that associations and properties are distinct in DDI-CDI, because it is based on a UML formalisation where such a distinction is important (associations never link to literal values). They are listed separately here, for ease of reference to the DDI-CDI documentation.

Data Store

This represents a queryable service.

Type: cdi:DataStore

Properties:

name (Required) – a string assigning a name to the service. More complete description of the service can be provided using the fields in the CDIF Discovery recommendations.

Associations:

has (Required, Multiple) – links the Data Store to a Logical Record.

Logical Record

This describes the set of variables available in the service or data set at a logical level.

Type: cdi:LogicalRecord

Properties:

None.

Associations:

- *organizes* (Optional, Multiple) – points to a Wide Data Set, Long Data Set, or Dimensional Data Set.
- *has* (Required, Multiple) – points to the Instance Variables making up the Logical Record.
- *isDefinedBy* (Optional, Multiple) – points to a Unit Type, Universe, or Population which provides a description of the subject of the Logical Record.

Wide Data Set

This represents a data set where each row in the table is a record, holding a set of values for variables, represented by columns.

¹² <https://w3c.github.io/json-ld-syntax/#string-internationalization>

Type: cdi:WideDataSet, cdi:PhysicalDataSet

Properties:

- *name* (Required) – a string value holding the name of the data set. May be provided in multiple, language-specific versions.
- *physicalFileName* (Required) - a URL providing the location of the data set.

Associations:

- *isStructuredBy* (Required) – points to an instance of cdi:WideDataStructure describing the structure of the data set.

Long Data Set

This represents a data set where each row in the table is one or more measurements, with another column for each measure in the row that indicates which logical variable is being measured. The variable being measured is specified for each row, and can vary from row to row.

Type: cdi:LongDataSet, cdi:PhysicalDataSet

Properties:

- *name* (Required) – a string value holding the name of the data set. May be provided in multiple, language-specific versions.
- *physicalFileName* (Required) - a URL providing the location of the data set.

Associations:

- *isStructuredBy* (Required) – points to an instance of cdi:LongDataStructure describing the structure of the data set.

Dimensional Data Set

This represents a data set where each measurement and associated attributes are defined by a set of dimensions. The data set can be understood as a “cube”. (See the DDI-CDI documentation for a more complete description.)

Type: cdi:DimensionalDataSet, cdi:PhysicalDataSet

Properties:

- *name* (Required) – a string value holding the name of the data set. May be provided in multiple, language-specific versions.
- *physicalFileName* (Required) - a URL providing the location of the data set.

Associations:

- *isStructuredBy* (Required) – points to an instance of `cdi:DimensionalDataStructure` describing the structure of the data set.

Wide Data Structure

This is a description of the structure of a Wide Data Set.

Type: `cdi:WideDataStructure`

Properties:

- None.

Associations:

- *has* (Required) – Points to the `cdi:PrimaryKey` used to identify records in the data set.
- *has* (Required, Multiple) – References the components (variables) which make up the records: links to `cdi:IdentifierComponent`, `cdi:AttributeComponent`, `cdi:MeasureComponent`.

Long Data Structure

This is a description of the structure of a Long Data Set.

Type: `cdi:LongDataStructure`

Properties:

- None.

Associations:

- *has* (Required) – Points to the `cdi:PrimaryKey` used to identify records in the data set.
- *has* (Required, Multiple) – References the components (variables) which make up the records: links to `cdi:IdentifierComponent`, `cdi:AttributeComponent`, `cdi:MeasureComponent`, `cdi:DescriptorComponent`, `cdi:VariableValueComponent`. Note that the components describe only the variables which are part of the presentation of the data set - other purely logical variables are referenced from the `LogicalRecord` associated with the Long Data Set and its components (see above).

Dimensional Data Structure

This is a description of the structure of a Dimensional Data Set.

Type: `cdi:DimensionalDataStructure`

Properties:

- None.

Associations:

- *has* (Required) – Points to the `cdi:PrimaryKey` used to identify records in the data set, made up of a set of `cdi:DimensionComponents`.
- *has* (Required, Multiple) – References the components (variables) which make up the records: links to `cdi:DimensionComponent`, `cdi:AttributeComponent`, `cdi:MeasureComponent`.
- *has* (Optional, Multiple) – Points to the `cdi:ComponentPosition` instances used to order the instances of `cdi:DimensionComponent` when the Primary Key is ordered.

Instance Variable

This represents a variable used in a data set. The full set of logical variables must always be described, so that every value in the data set is accounted for.

Type: `cdi:InstanceVariable`, `cdi:RepresentedVariable` [only if Represented Variables are not instantiated separately]

Properties:

- *name* (Required) – a string value holding the name of the variable. May be provided in multiple, language-specific versions.
- *definition* (Required) – a string value holding the definition of the variable. May be provided in multiple, language-specific versions.
- *describedUnitOfMeasure* (Optional) - a specification of the UOM, specified according to the “defined term” implementation pattern specified in Appendix 1.
- *unitOfMeasureKind* (Optional) - a specification of the quantity-kind for the UOM, specified according to the “defined term” implementation pattern specified in Appendix 1.

Associations:

- *measures* (Optional) – a link to an instance of `cdi:Population`.
- *takesSubstantiveValuesFrom* (Required) - a link to an instance of `cdi:EnumeratedValueDomain`, which in turn provides information about the typing of the recorded values for the variable.

- *takesSentinelValuesFrom* (Optional) - a link to an instance of `cdi:SentinelvalueDomain`, which in turn provides information about how flags, missing values, and similar notations are typed in the variable.

Represented Variable

This represents a reusable variable description.

Type: `cdi:RepresentedVariable`

Properties:

- *name* (Required) – a string value holding the name of the variable. May be provided in multiple, language-specific versions.
- *definition* (Required) – a string value holding the definition of the variable. May be provided in multiple, language-specific versions.
- *describedUnitOfMeasure* (Optional) - a specification of the UOM, specified according to the “defined term” implementation pattern specified in Appendix 1.
- *unitOfMeasureKind* (Optional) - a specification of the quantity-kind for the UOM, specified according to the “defined term” implementation pattern specified in Appendix 1.

Associations:

- *measures* (Optional) – a link to an instance of `cdi:Universe`.
- *takesSubstantiveValuesFrom* (Required) - a link to an instance of `cdi:SubstantiveValueDomain`, which in turn provides information about the typing of the recorded values for the variable.
- *takesSentinelValuesFrom* (Optional) - a link to an instance of `cdi:SentinelvalueDomain`, which in turn provides information about how flags, missing values, and similar notations are typed in the variable.

Reference Variable

Reference Variables are presentational variables used only in the description of Long Data Sets. They contain values from the set of logical variables held in the data, and must have a representation which supports the full range of possible values from them.

Type: `cdi:ReferenceVariable`

Properties:

- *Name* (Required) – a string value holding the name of the variable. May be provided in multiple, language-specific versions.

Associations:

- *takesSubstantiveValuesFrom* (Required) - a link to an instance of `cdi:SubstantiveValueDomain`, which in turn provides information about the typing of the recorded values for the variable.
- *takesSentinelValuesFrom* (Optional) - a link to an instance of `cdi:SentinelvalueDomain`, which in turn provides information about how flags, missing values, and similar notations are typed in the variable.

Descriptor Variable

Descriptor Variables hold values which reference the logical variables in the data set, indicating which one the associated value in the corresponding Reference Variable is a measure/value for. Descriptor Variables are presentational variables found only in Long Data Sets.

Type: `cdi:DescriptorVariable`

Properties:

- *Name* (Required) – a string value holding the name of the variable. May be provided in multiple, language-specific versions.

Associations:

- *takesSubstantiveValuesFrom* (Required) - a link to an instance of `cdi:DescriptorValueDomain`, which in turn provides information about logical variables to which each enumerated value corresponds, using the `cdi:Descriptor` class.

Substantive Value Domain

Substantive values reflect the definition of the variable which they populate, as opposed to being an indication of a process or system status (“missing,” etc.). They may be enumerated or otherwise typed.

Type: `cdi:SubstantiveValueDomain`

Properties:

- None.

Associations:

- *recommendedDataType* (Required) - provides a link either to a skos:ConceptScheme (for enumerated values) or an XSD data type (see list in the text above). This describes the datatype of the values permitted as valid for populating the fields described by the variable.

Sentinel Value Domain

Sentinel values are those which indicate some process or system status or value which is not directly meaningful in terms of the variable definition of the field they populate. A typical example is flags for missing values.

Type: cdi:SentinelValueDomain

Properties:

- None.

Associations:

- *recommendedDataType* (Required) - provides a link either to a skos:ConceptScheme (for enumerated values) or an XSD data type (see list in the text above). This describes the datatype of the sentinel values permitted as valid for populating the fields described by the variable.

Descriptor Value Domain

This provides a list of instances of the Descriptor class, connecting the values found in a Long Data Set's Descriptor Variable with the logical variables for which values are provided in the Reference Variable.

Type: cdi:DescriptorValueDomain

Properties:

- None.

Associations:

- None.

Descriptor

An instance of the cdi: Descriptor class

Type: cdi:Descriptor

Properties:

- *content* (Required) - the value appearing in the Long Data Set, expressed as a string.

Associations:

- *identifies* (Required) – reference to the `cdi:InstanceVariable` which corresponds to the *content* property.

Unity Type

A Unit Type is the object or phenomenon serving as the subject of a data record, identified at a general level. If the values in a data set are “Age,” “Sex,” and “Occupation” the Unit Type would be “person”. If the values are a measure of salinity and depth, the Unit Type might be “sea water”. The value of an instance of `cdi:UnitType` is the general term for the unifying subject of a data record (an instance of `cdi:LogicalRecord`).

Type: `cdi:UnitType`

Properties:

- *name* (Required) – a string value holding the name of the variable. May be provided in multiple, language-specific versions.
- *descriptiveText* (Optional) – a textual description of the Unit Type. May be provided in multiple, language-specific versions.

Associations:

- None.

Universe

An instance of `cdi:Universe` qualifies a Unit Type, to provide a more-specific description of the subject of the data, but excluding time and geography. If the Unit Type is “person,” the Universe might be “adults”. If the Unit Type is “sea water,” the Universe might be “sea water in temperate oceans”.

Type: `cdi:Universe`

Properties:

- *name* (Required) – a string value holding the name of the variable. May be provided in multiple, language-specific versions.
- *descriptiveText* (Optional) – a textual description of the Unit Type. May be provided in multiple, language-specific versions.

Associations:

- *uses* (Optional) – a reference to the instance of `cdi:UnitType` which the Universe qualifies.

Population

This is a fully qualified description of the subject of the data in a field or record. It extends the description provided by Unit Type and Universe by adding geographical and temporal bounds.

Type: `cdi:Population`

Properties:

- *name* (Required) – a string value holding the name of the variable. May be provided in multiple, language-specific versions.
- *descriptiveText* (Optional) – a textual description of the Unit Type. May be provided in multiple, language-specific versions.

Associations:

- *uses* (Optional, 2 Maximum) – a reference to the instance of `cdi:Universe` and/or `cdi:UnitType` which the Universe qualifies.

Identifier Component

The instance of `cdi:IdentifierComponent` is a reference to a variable which provides a unique identifier for the case being observed in the Logical Record. It is typically an assigned or official identifier for the unit, according to an external scheme (e.g., a taxpayer ID for a person, a sample identifier, etc.). The Identifier Component is a role attached to the variable to describe its function within the data structure - it is usually part of (and may be the entire) key for the record.

Type: `cdi:IdentifierComponent`

Properties:

- None.

Associations:

- *isDefinedBy* (Required) – A reference to an instance of `cdi:InstanceVariable` (or, if present, `cdi:RepresentedVariable`) which defines the Identifier Component variable.

Measure Component

The instance of `cdi:MeasureComponent` indicates that the referenced variable contains a measurement or observation value, or a derived value, within the Logical Record. Note that `cdi:MeasureComponent`

might be a simple value (string, number, term), or an object (e.g. an array of spectral values). For this profile, we are restricting discussion to simple values.

Type: cdi:MeasureComponent

Properties:

- None.

Associations:

- *isDefinedBy* (Required) – A reference to an instance of cdi:InstanceVariable (or, if present, cdi:RepresentedVariable) which defines the Measure Component variable.

Attribute Component

Each instance of cdi:AttributeComponent indicates that the referenced variable holds a value which describes the values in the Measure Components or otherwise provides information which is not itself a measure/observation or derived value.

Type: cdi:AttributeComponent

Properties:

- None.

Associations:

- *isDefinedBy* (Required) – A reference to an instance of cdi:InstanceVariable (or, if present, cdi:RepresentedVariable) which defines the Attribute Component variable.

Dimension Component

The instances of cdi:DimensionComponent are references to the variables which are the components of a compound identifier, in which each variable is a single field - an axis - in a coordinate system addressing a location in a matrix. These variables are often categorical, but also commonly include time. Unlike other Components, they may be ordered using instances of cdi:ComponentPosition.

Type: cdi:DimensionComponent

Properties:

- None.

Associations:

- *isDefinedBy* (Required) – A reference to an instance of `cdi:InstanceVariable` (or, if present, `cdi:RepresentedVariable`) which defines the Attribute Component variable.

Component Position

Each instance of `cdi:ComponentPosition` indexes a Dimension Component within the Primary Key of a Dimensional Data Structure, by assigning an incrementing *value* to it (lowest number comes first in the sequence).

Type: `cdi:ComponentPosition`

Properties:

- *value* (Required) - An integer indicating the position of the Dimension Component within the sequence of the Primary key for the referencing Dimensional Data Structure.

Associations:

- *indexes* (Required) – a reference to the instance of `cdi:DimensionComponent` which is being ordered. Assigns the *value* to it as a position within the Primary Key.

Primary Key

The instance of Primary Key defines the set of Components which, taken together, provide the identifying values for a single logical record within a data structure. The values of the Primary Key are used to reference the non-Primary Key fields within the Logical Record, by additionally specifying the Component/Instance Variable wanted. Such fields include the Measure Components and non-identifying Attribute Components.

Type: `cdi:PrimaryKey`

Properties:

- None.

Associations:

- *isComposedOf* (Required, Multiple) – References to the instances of `cdi:PrimaryKeyComponent` making up the Primary Key.

Primary Key Component

Instances of `cdi:PrimaryKeyComponent` reference the Components which make up the primary Key in a Data Structure.

Type: `cdi:PrimaryKeyComponent`

Properties:

- None.

Associations:

- *correspondsTo* (Required) – A reference to the instance of cdi:IdentifierComponent, cdi:AttributeComponent, cdi:DimensionComponent, or cdi:VariableDescriptorComponent used as a Primary Key Component.

Variable Value Component

Each instance of a cdi:VariableValueComponent points to a Reference Variable, which holds the values of several logical variables combined into a single field within a Long Data Structure. This Component is part of a presentational structure, as opposed to a logical one.

Type: cdi:VariableValueComponent

Properties:

- None.

Associations:

- *isDefinedBy* (Required) – A reference to an instance of cdi:ReferenceVariable.

Variable Descriptor Component

Instances of cdi:VariableDescriptorComponent are fields in a Long Data Structure which indicate the logical variable with which the value of the Variable Value Component/Reference Variable are associated.

Type: cdi:VariableDescriptorComponent

Properties:

- None.

Associations:

- *refersTo* (Required) – A reference to an instance of cdi:DescriptorVariable.

Physical Segment Layout

Each instance of cdi:PhysicalSegmentLayout describes the physical expression of a record in a data file or other storage expressed as a textual format. The Logical Record encoded is referenced. There is only a

single Physical Segment Layout used to describe data files - all records are assumed to have the same layout.

Type: cdi:PhysicalSegmentLayout

Properties:

- *arrayBase* (Optional) - This is a literal value of either “0” or “1” indicating the starting number for the sequence of variables in the record as indexed by instances of cdi:ValueMappingPosition for each variable.
- *commentPrefix* (Optional) - A string used to indicate that a line in the input is a comment.
- *delimiter* (Optional) - The character used to separate fields in a delimited file (for example, the comma in a CSV file),
- *encoding* (Required) - A string specifying the character encoding (typically UTF-8). Values are the same as those used for the HTML *charset* attribute.¹³
- *escapeCharacter* (Optional) - Provides the string which is used to escape the quote character.
- *hasHeader* (Optional) - A boolean value indicated as “true” or “false” (case sensitive) which indicates that the number of initial rows in *headerRowCount* should be skipped, as they do not contain data values.
- *headerRowCount* (Optional) - An integer which indicates how many initial rows to skip as containing header information. Only used if *hasHeader* has a value of “true”.
- *isDelimited* (Optional) - A boolean value indicated as “true” or “false” (case sensitive) which says whether the file is delimited. If “true”, then the *delimiter* property will provide the delimiter. If “true” then *isFixedWidth* must have a value of “false” if provided.
- *isFixedWidth* (Optional) - A boolean value indicated as “true” or “false” (case sensitive) which says whether the file has fixed-width fields. If “true” then *isDelimited* must have a value of “false” if provided. The starting and ending positions of each field are provided by the instances of cdi:ValueMapping associated with each variable in the record, in relation to the *arrayBase* property.
- *lineTerminator* (Optional) - The character sequence used to terminate rows. Defaults to the array [CRLF, LF] as per the Metadata Vocabulary for Tabular Data.¹⁴

¹³ See https://www.w3schools.com/html/html_charset.asp.

¹⁴ <https://www.w3.org/TR/tabular-metadata/>

- *quoteCharacter* (Optional) - The string that is used around escaped cells, or null, set by the *quoteChar* property of a dialect description. The default is ".". See W3C Recommendation "Model for Tabular Data and Metadata on the Web".¹⁵
- *skipInitialSpace* (Optional) - A boolean value expressed as "true" or "false" (case-sensitive) indicating that - if "true" - white space at the beginning of a row or following a delimiter should be ignored.
- *trim* (Optional) - This field takes the values "Both", "End", "Neither", and "Start" describing the places where white space can be trimmed from rows. Default is "Neither".

Associations:

- *Formats* (Required) - specifies an instance of *cdi:LogicalRecord*.
- *has* (Required, Multiple) – A reference to an instance of *cdi:ValueMapping*.
- *has* (Required, Multiple) - A reference to an instance of *cdi:ValueMappingPosition*, one per Value Mapping.

Value Mapping

Instances of *cdi:ValueMapping* are used to indicate the connection of variables with a Physical Segment layout.

Type: *cdi:ValueMapping*

Properties:

- *decimalPositions* (Optional) - An integer indicating the number of decimal positions when the decimal separator is implied.
- *defaultDecimalSeparator* (Optional) - A string providing the character used as a decimal separator.
- *length* (Optional) - An integer specifying the length of a fixed-width field.
- *maxLength* (Optional) - An integer indicating the maximum length of the field value.
- *minimumLength* (Optional) - An integer expressing the maximum length of the field value

Associations:

¹⁵ <https://www.w3.org/TR/tabular-data-model/#parsing>

- *formats* (Required) - specifies an instance of cdi:InstanceVariable, cdi:RepresentedVariable, cdi:ReferenceVariable, or cdi:DescriptorVariable.

Value Mapping Position

Instance of cdi:ValueMappingPosition provides the sequence of the mapped variables within the Physical Segment Layout.

Type: cdi:ValueMappingPosition

Properties:

- *value* (Required) – An integer indicating the relative position of the mapped variable in the sequence of the Physical Segment Layout. Uses the *arrayBase* property of the Physical Record Segment as a starting point.

Associations:

- *indexes* (Required) – A reference to an instance of cdi:ValueMapping